MY LITTLE RPG - OPTIMISATION

DATE DE REMISE : VOIR LÉA **DURÉE : 1 SE**

OBJECTIFS DE CE TRAVAIL

- Repayer la dette technique accumulée dans les 4 premières étapes
- Améliorer les performances
- Renforcer la robustesse
- Sécuriser les accès

CONSIGNES / CONTRAINTES

Balise IAg CChic niveau 0 : Interdiction d'utiliser l'IAg



CRITÈRES D'ÉVALUATION

- ✓ Amélioration du projet sous tous les aspects mentionnés
- ✓ Authenticité de la démarche de d'amélioration
- ✓ Collaboration à l'équipe

©2025 RTL, PAS

DESCRIPTION

Après 4 étapes, votre application a vécu plusieurs ajouts et modifications. Dans l'objectif de remettre un travail fonctionnel, vous avez certainement dû faire des choix qui révèlent maintenant leur manque de sagesse. De plus, dans ma correction, j'ai vu de nombreuses bévues au niveau de la performance, de la robustesse et de la sécurité qu'il faudrait améliorer.

Avant d'entamer la deuxième moitié de la session, nous allons donner un bon coup de balai dans notre code. **Faire cet entretien va briser certaines fonctionnalités. C'est normal, et il faut les réparer en respectant les bonnes pratiques de programmation.** L'objectif est, pour chacun des enjeux, d'améliorer la conception de notre MyLittleRPG.

EXEMPLES D'AMÉLIORATIONS À APPORTER

Autant dans votre API que dans votre site Web, passez tout le code en revue en gardant en tête les quatre enjeux : Refactorisation, Performance, Robustesse, Sécurité. Aidez vous des exemples ci-dessous pour cibler vos améliorations.

REFACTORISATION

- Ajouter des DTO aux endroits adéquats
- Éliminer la duplication de code
- Rendre plus lisible le code
    - lignes complexes
    - noms de fonctions et de variables génériques ou mal choisis
    - nombres magiques
    - fonctions trop longues
- Standardiser la nomenclature
- Réduire le nombre de classes et de fichiers
- Éliminer les « folleries » de l'IAg
    - Fonctions inutiles
    - Vérification à outrance

©2025 RTL, PAS

    - Minimisation de la structure surchargée que ces outils génèrent

PERFORMANCE

- Diminuer les accès à la base de données.
- Ajouter des conditions qu'on peut vérifier pour mettre fin hâtivement aux traitements lourds.
- Choisir judicieusement où traiter une information : dans la BD, dans l'API ou dans la page Web. Migrer la fonctionnalité au besoin.

ROBUSTESSE

- Scruter la validation des données pour éviter des valeurs invalides. Ajouter des blocs try/catch et faire une meilleure gestion des messages d'erreur.
- Vérifier l'usage des codes d'erreur HTTP :
    - Chaque fonction de l'API retourne le bon code de réponse.
    - Chaque appel à l'API traite tous les codes de réponse possibles.

SÉCURITÉ

- S'assurer de l'utilité et de la sécurité de tous les endpoints, et désactiver ceux qui sont inutiles ou non-sécurisés.
- Vérifier l'authentification de l'utilisateur pour les opérations concernant son compte ou son personnage.
- Éliminer le transfert superflu d'information sensible.
- Renforcer l'application contre les tentatives d'usage illicite.

©2025 RTL, PAS

À REMETTRE

Faites une branche dans votre dépôt GIT qui contiendra la version que vous remettez pour cette étape.

Remettez :

- Un document clair indiquant TOUTES les modifications que vous avez apportées pour cette étape du projet.
    - Pour chaque classe ou fichier
        - Quelle fonction a été ajoutée/supprimée/modifiée
        - Pour chaque changement dans la fonction, mentionner l'enjeu qui était en cause et la justification du changement.

©2025 RTL, PAS